

LogiCube



De l'eau !

Première étape :

WaterGeneration.cs

- Créez un nouveau fichier WaterGeneration.cs
- Ce script va contenir la logique de l'eau
- Cette classe n'hérite pas de MonoBehaviour : ce n'est pas un composant Unity, mais une classe utilitaire .
- On ajoute [System.Serializable]
- Cela permet à Unity d'afficher les variables dans l'Inspector

```
using System.Collections.Generic;
using UnityEngine;

[System.Serializable]
public class WaterGeneration
{
    public int waterLevel = 8;

    public bool IsWaterBlock(int y, int terrainHeight)
    {
        return y > terrainHeight && y <= waterLevel;
    }

    public void AddWaterColumn(Dictionary<Vector3Int, BlockType> blocks, int x, int z, int terrainHeight)
    {
        if (terrainHeight >= waterLevel)
            return;

        for (int y = terrainHeight + 1; y <= waterLevel; y++)
            blocks[new Vector3Int(x, y, z)] = BlockType.Water;
    }

    public bool IsUnderWater(int groundHeight)
    {
        return groundHeight < waterLevel;
    }
}
```

Première étape :

WaterGeneration.cs

- waterLevel
 - définit la hauteur maximum de l'eau dans le monde
- IsWaterBlock(int y, int terrainHeight)
 - vérifie si un bloc doit être de l'eau
 - retourne true si le bloc est au-dessus du sol
 - retourne true si le bloc est encore sous le niveau de l'eau
- AddWaterColumn(...)
 - ajoute une colonne d'eau à la position (x, z)
 - ne fait rien si le terrain est déjà au-dessus du niveau de l'eau
 - remplit tous les blocs entre terrainHeight + 1 et waterLevel
- IsUnderWater(int groundHeight)
 - vérifie si le sol est sous le niveau de l'eau
 - utile pour savoir si une zone est immergée

```
using System.Collections.Generic;
using UnityEngine;

[System.Serializable]
public class WaterGeneration
{
    public int waterLevel = 8;

    public bool IsWaterBlock(int y, int terrainHeight)
    {
        return y > terrainHeight && y <= waterLevel;
    }

    public void AddWaterColumn(Dictionary<Vector3Int, BlockType> blocks, int x, int z, int terrainHeight)
    {
        if (terrainHeight >= waterLevel)
            return;

        for (int y = terrainHeight + 1; y <= waterLevel; y++)
            blocks[new Vector3Int(x, y, z)] = BlockType.Water;
    }

    public bool IsUnderWater(int groundHeight)
    {
        return groundHeight < waterLevel;
    }
}
```

BlockType

- On ajoute donc un nouveau bloc "Water" et des fonctions utiles pour le reconnaître plus facilement dans le code.

```
public enum BlockType
{
    Air = 0,    // vide
    Dirt = 1,   // terre
    Stone = 2,  // pierre
    Grass = 3,  // herbe
    Tnt = 4,
    Water = 5,
}

public static class BlockTypeExtensions
{
    public static bool IsSolid(this BlockType type) => type != BlockType.Air;
    public static bool IsRenderable(this BlockType type) => type != BlockType.Air;
    public static bool IsWater(this BlockType type) => type == BlockType.Water;
}
```

BlockType

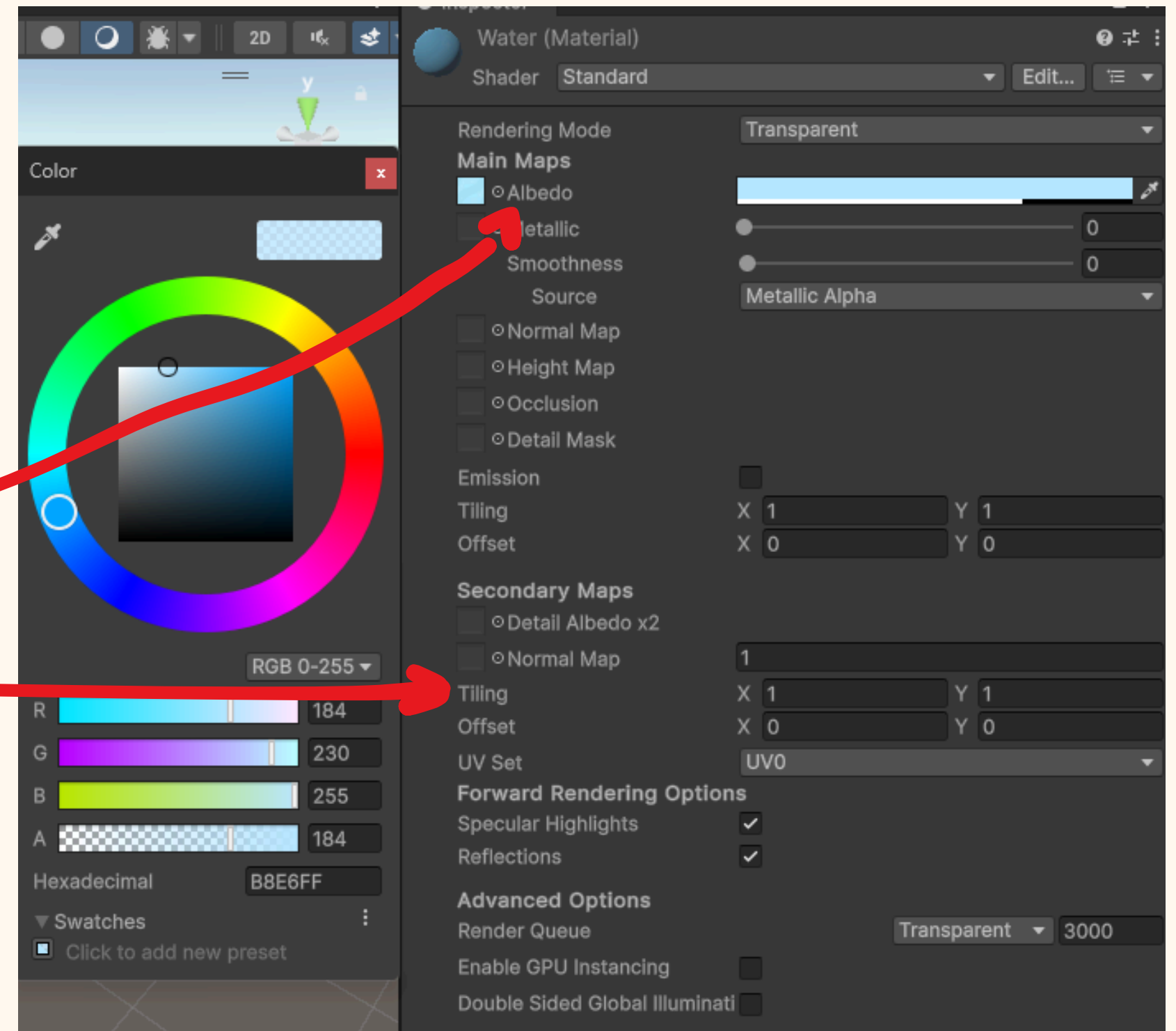
- On ajoute donc un nouveau bloc "Water" et des fonctions utiles pour le reconnaître plus facilement dans le code.

```
public enum BlockType
{
    Air = 0,    // vide
    Dirt = 1,   // terre
    Stone = 2,  // pierre
    Grass = 3,  // herbe
    Tnt = 4,
    Water = 5,
}

public static class BlockTypeExtensions
{
    public static bool IsSolid(this BlockType type) => type != BlockType.Air;
    public static bool IsRenderable(this BlockType type) => type != BlockType.Air;
    public static bool IsWater(this BlockType type) => type == BlockType.Water;
}
```


Water material

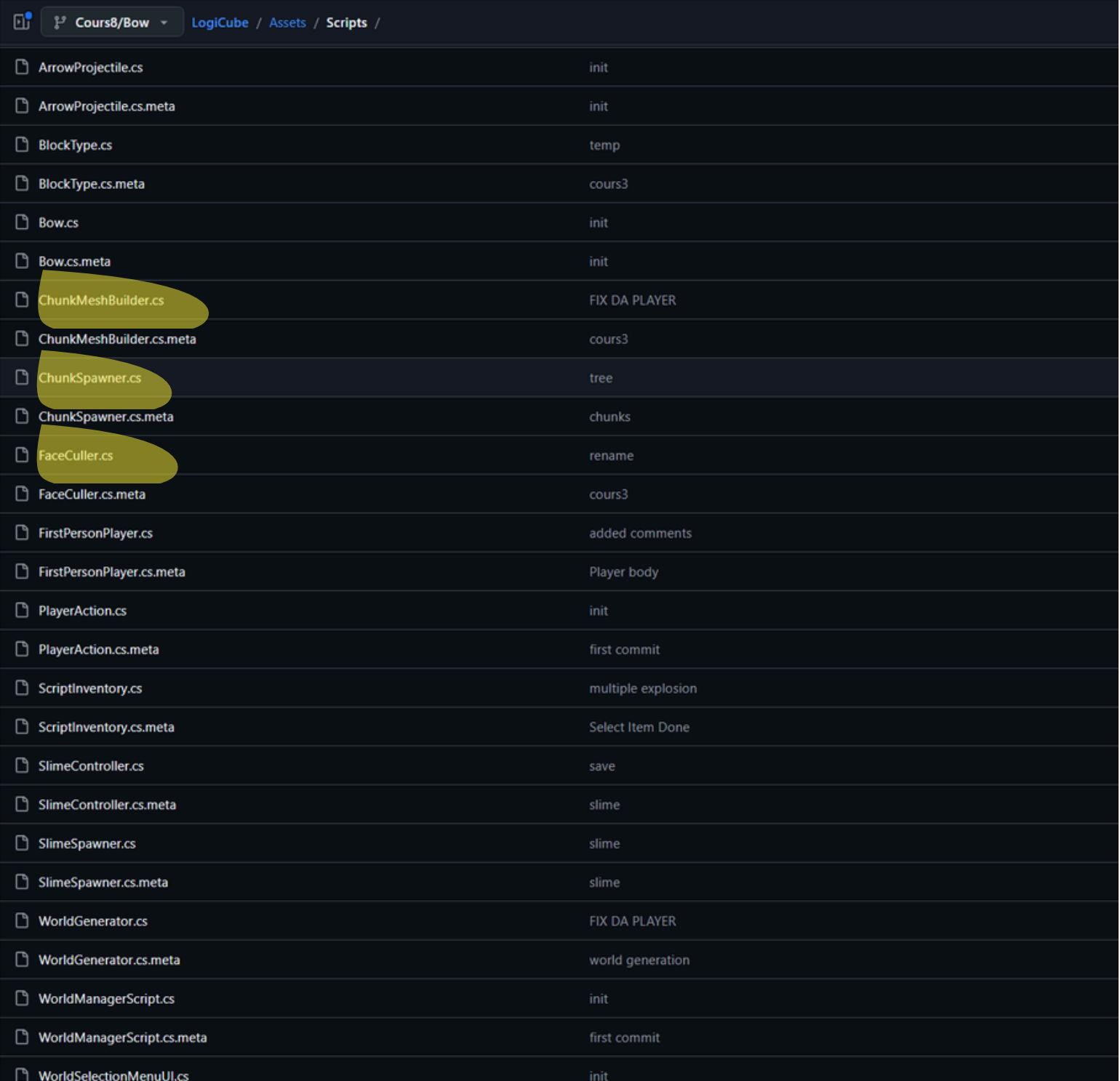
- Créer un nouveau material: water.mat
- Dans l'Inspector, vérifier que le shader est Standard
- Changer Rendering Mode en Transparent
- Albedo → mettre water.png
- Baisser l'alpha de la couleur (A) pour rendre l'eau transparente
- modifier le Tiling



Copier-coller les scripts

ChunkSpawner, ChunkMeshBuilder et FaceCuller

- Copier-coller les scripts ChunkSpawner, ChunkMeshBuilder et FaceCuller
- Les ajouts faits dans ces scripts sont surtout des modifications de performance
- Leur but est d'optimiser l'affichage et la gestion du monde
- Ces parties sont plus techniques et moins importantes pour comprendre la génération de l'eau



ArrowProjectile.cs	init
ArrowProjectile.cs.meta	init
BlockType.cs	temp
BlockType.cs.meta	cours3
Bow.cs	init
Bow.cs.meta	init
ChunkMeshBuilder.cs	FIX DA PLAYER
ChunkMeshBuilder.cs.meta	cours3
ChunkSpawner.cs	tree
ChunkSpawner.cs.meta	chunks
FaceCuller.cs	rename
FaceCuller.cs.meta	cours3
FirstPersonPlayer.cs	added comments
FirstPersonPlayer.cs.meta	Player body
PlayerAction.cs	init
PlayerAction.cs.meta	first commit
ScriptInventory.cs	multiple explosion
ScriptInventory.cs.meta	Select Item Done
SlimeController.cs	save
SlimeController.cs.meta	slime
SlimeSpawner.cs	slime
SlimeSpawner.cs.meta	slime
WorldGenerator.cs	FIX DA PLAYER
WorldGenerator.cs.meta	world generation
WorldManagerScript.cs	init
WorldManagerScript.cs.meta	first commit
WorldSelectionMenuUI.cs	init

Nouveau WorldGenerator : vue d'ensemble

- Avant, la hauteur du terrain était calculée avec un seul bruit de Perlin
- Cela créait surtout des collines et des creux
- Il y avait peu de grandes mers ou de vrais continents
- Maintenant, on combine :
 - un bruit de Perlin pour les grandes formes du monde
 - un bruit de Perlin pour les petits détails du relief
 - un niveau de mer pour ajouter l'eau
- Résultat : un terrain plus naturel avec des zones basses, des côtes et des océans

Le script nouveau WorldGenerator calcule d'abord la hauteur du sol, puis ajoute l'eau si le terrain est sous le niveau de la mer.

Les nouveaux paramètres

- chunkSize
 - taille de la zone générée
- noiseScale
 - taille des petits détails du terrain
- terrainHeight
 - intensité des petits reliefs
- continentNoiseScale
 - taille des grandes formes du monde
- continentHeight
 - intensité des continents et des océans
- detailStrength
 - quantité de détails ajoutés au relief
- terrainBaseOffset
 - décale la hauteur globale du monde
- waterGeneration
 - contient les paramètres de l'eau
- Water
 - raccourci pratique pour accéder à waterGeneration

```
public class WorldGenerator : MonoBehaviour
{
    public int chunkSize = 32;
    public float noiseScale = 0.1f;
    public int terrainHeight = 10;
    public float continentNoiseScale = 0.008f;
    public int continentHeight = 10;
    [Range(0f, 1f)] public float detailStrength = 0.3f;
    public int terrainBaseOffset = -1;
    public WaterGeneration waterGeneration = new();
    public WaterGeneration Water => waterGeneration ??= new();
}
```

fonctions utilitaire

- GetSeedOffset()
 - génère un décalage aléatoire à partir de la seed
 - permet d'obtenir un monde différent pour chaque seed
 - cette logique existait déjà avant, mais elle est maintenant isolée dans une fonction
- SampleNoise()
 - calcule une valeur de bruit de Perlin à partir de x, z, offset et scale
 - évite de réécrire plusieurs fois le même calcul
 - logique existait aussi
- GetTerrainHeight()
 - calcule la hauteur finale du terrain
 - combine une grande noise pour les continents
 - combine une petite noise pour les détails
 - ajoute le niveau de mer comme base
 - retourne la hauteur finale du sol

```
public Vector2 GetSeedOffset(int seed)
{
    var prng = new System.Random(seed);
    return new Vector2(prng.Next(-100000, 100000),
        prng.Next(-100000, 100000));
}

private float SampleNoise(int x, int z, Vector2 offset, float scale)
{
    float nx = (x + offset.x) * scale;
    float nz = (z + offset.y) * scale;
    return Mathf.PerlinNoise(nx, nz);
}
```

```
public int GetTerrainHeight(int x, int z, Vector2 offset)
{
    float continent = SampleNoise(x, z, offset, continentNoiseScale);
    float detail = SampleNoise(x, z, offset, noiseScale);

    float height = Water.waterLevel + terrainBaseOffset;
    height += (continent - 0.5f) * 2f * continentHeight;
    height += (detail - 0.5f) * 2f * terrainHeight * detailStrength;

    return Mathf.Max(0, Mathf.RoundToInt(height));
}
```

Generate

- Generate():
 - rappel : La fonction Generate parcourt le sol colonne par colonne et décide quoi placer à chaque position.
 - ajoutez cette ligne
 - Elle est exécutée après avoir généré le sol
 - La fonction AddWaterColumn regarde si la hauteur du terrain est sous le niveau de la mer
 - Si le terrain est trop bas, elle ajoute des blocs d'eau au-dessus du sol
 - Si le terrain est déjà assez haut, elle ne fait rien
 - Cette ligne remplit donc les zones basses avec de l'eau.

```
public Dictionary<Vector3Int, BlockType> Generate(int seed)
{
    Vector2 offset = GetSeedOffset(seed);

    var blocks = new Dictionary<Vector3Int, BlockType>();
    int half = chunkSize / 2;

    for (int x = -half; x < half; x++)
        for (int z = -half; z < half; z++)
        {
            int height = GetTerrainHeight(x, z, offset);

            for (int y = 0; y <= height; y++)
            {
                BlockType type;

                if(y <=3){
                    type = BlockType.Stone;
                }
                else if (y==height){
                    type = BlockType.Grass;
                }
                else{
                    type = BlockType.Dirt;
                }

                blocks[new Vector3Int(x, y, z)] = type;
            }

            Water.AddWaterColumn(blocks, x, z, height);
        }

    return blocks;
}
```